

Can LLMs only talk? Experimental studies on task scheduling with Large Language Models

Mengjuan Li^{†*}, Zhengguang Chen^{†*}, Huan Zhou^{†✉}, Zhipeng Wang[†],
Yingwen Chen[†], Baokang Zhao[†], Xue Ouyang[†], Jinshu Su^{†‡}

[†] National University of Defence Technology, Changsha, China

[‡] Academy of Military Science, Beijing, China

limj@nudt.edu.cn, czg85910@163.com, huanzhou@nudt.edu.cn, wangzp20@nudt.edu.cn,

ywch@nudt.edu.cn, bkzhao@nudt.edu.cn, ouyangxue@nudt.edu.cn, sjs@nudt.edu.cn

Abstract—Large Language Models (LLMs) have emerged as a disruptive technology for Natural Language Processing (NLP), achieving success in NLP-related generative applications. However, the potential capability of LLMs in other domains remains largely unexplored. To explore the potential of task scheduling with LLMs, we model a typical task scheduling scenario in cloud computing and transfer scheduling problems as natural language prompts. Afterward, the knowledge and reasoning abilities of LLMs are enabled to generate scheduling decisions. Six well-known and open-source LLMs are integrated into our framework to perform experimental studies, and the results are evaluated from multiple perspectives and compared with each other. Besides, traditional heuristic algorithms and a basic Reinforcement Learning (RL) method are all performed for comparison. Our results demonstrate: 1) compared to most heuristic methods, the decisions made by LLMs achieve better scheduling performance; 2) compared to the basic RL method, LLMs exhibit better generalization on various workload patterns; 3) the larger parameter size of the LLMs has, the better scheduling performance it achieves. To the best of our knowledge, our experimental study is the first exploration to apply LLMs in task scheduling. Our findings highlight the promising potential of LLMs as a novel approach to task scheduling, offering new avenues for research and practice.

Index Terms—task scheduling, LLMs, experimental study, prompt

I. INTRODUCTION

The research community has shown a heightened interest in large language models (LLMs), largely driven by the recent success of ChatGPT. By pre-training on extensive corpora, LLMs have demonstrated strong capabilities in addressing various Natural Language Processing (NLP) tasks. Currently, numerous pre-trained models with exceptional performance have emerged in the industry, such as GPT-4, Claude-3, and Llama-3, which can be adapted to new scenarios with minimal manual intervention and domain expertise. Specifically, by fine-tuning [1] or prompting [2], they can effectively tackle various downstream tasks.

LLMs have shown significant potential across various research domains. For instance, in the traditional NLP field, LLMs are effectively utilized for sequence labeling, information extraction, and text generation [3] [4] [5]. In the medical field, LLMs are widely employed for medical diagnosis and health analysis [6] [7]. Similarly, LLMs have been successfully utilized in the legal, financial, and educational sectors [8] [9]

[10]. LLMs' robust information extraction and generation capabilities significantly alleviate the workload of professionals in these fields. Furthermore, in the domain of networking and computing, LLMs have proven valuable in applications such as vulnerability detection and automated configuration generation [11] [12].

However, we have noted that the applications of LLMs appear to be concentrated in the NLP and CV domains. Even within specialized fields, tasks handled by LLMs largely exhibit a text-generation-based pattern. Here, "text" is used broadly, encompassing code, numbers, protocols, and other symbolic forms, which leads to a key question: Can LLMs only talk? Are LLMs restricted to tasks that are strongly generation-related? Can they directly handle more complex, non-generative tasks, such as decision-making, i.e., task scheduling? To investigate the above issue, we conducted an experimental study on the performance of LLMs in the context of task scheduling.

It is well known that scheduling is a core issue in cloud computing, appearing in various contexts such as cluster scheduling, resource management, and task mapping. Effective scheduling strategies directly influence the quality of task completion within computational systems. Scheduling typically manifests as an NP-hard online decision-making problem. Given the complexity and non-generative nature of task scheduling, it is worth to investigate whether the large-scale pre-trained foundation models can be capable of dealing with the task scheduling.

Our innovative exploration of scheduling with LLMs brings in several benefits. First, it provides a more user-friendly and accessible solution for non-research users. Additionally, our investigation demonstrates the potential of LLMs to address complex, non-generative problems. The exploration may also inspire further research directions and lead to the development of novel applications.

Applying LLMs to task scheduling, as a non-generative task, presents the primary challenge of translating the problem into a natural language description. As the first empirical study to apply LLMs to task scheduling, we adopted a simple yet effective experimental setup. We selected the online multi-resource cluster scheduling as the case study for simulation experiments. A template-based approach was used to con-

vert the scheduling problem into a textual format, which is understandable by LLMs. Additionally, to evaluate the basic scheduling performance of general LLMs, we employed a prompt-based approach to enable LLMs to perform inference within the task scheduling context, without extra training or any fine-tuning.

We first extracted the available resources and job requirements based on the state of the multi-resource cluster environment and converted the information into textual descriptions. Using the Langchain framework, prompts were generated and fed into the LLMs. The outputs from the LLMs were then parsed to extract the next scheduling strategy, which was used to update the available resources and the pending job queue. The above process was repeated until all jobs were completed. Experiments were conducted with several well-known and open-source LLMs and compared against traditional heuristic algorithms and a basic Reinforcement Learning (RL) method. Our key contributions are summarized as follows:

- We explore the potential of task scheduling with Large Language Models, representing the first academic attempt to the best of our knowledge. Specifically, available resources and task sequences are converted into textual descriptions, which are then input into LLMs through prompts to infer scheduling decisions.
- A comparison was conducted between LLM-based scheduling and traditional heuristic methods as well as a basic RL model. Experimental results show that LLM-based scheduling outperforms most traditional heuristic algorithms and exhibits better generalization capabilities compared to the RL model.
- We conduct experimental comparisons among six popular open-source LLMs available in the market, including Llama3-8B, Llama3.1-8B, Qwen2-7B, Qwen2.5-7B, Gemma2-9B, and Mistral-7B. The analysis reveals a positive correlation between the parameter size of LLMs and their performance on task scheduling.

The rest of this paper is organized as follows. Section II introduces the background for this study, covering both task scheduling and large language models. Section III details the experimental design. We report on the results of the experimental study in Section IV. Finally, we conclude this paper in Section V.

II. BACKGROUND

In this section, we introduce the task scheduling problem and existing related approaches. We then present background knowledge related to large language models, including the necessary tools utilized in this study and current popular open-source LLMs.

A. Task scheduling

The exponential growth of data and computational requirements has heightened the demand for reliable cloud services [13] [14]. Task scheduling, a cornerstone of cloud computing, plays a crucial role in balancing user satisfaction and provider profitability. By optimizing resource allocation,

effective scheduling algorithms can ensure high quality of service (QoS) for users and improve the economic efficiency of cloud providers.

Traditional solutions for task scheduling typically involve designing various heuristic algorithms. The primary objective of heuristic algorithms is to obtain optimal solutions without exhaustively searching the solution space. Consequently, they can be adapted to different operational contexts with relatively minor modifications. However, in distributed computing, factors such as workload dynamically fluctuate, making it difficult for accurate modeling. Furthermore, as environmental conditions change, heuristic algorithms often require redesign, which is a complex process.

Beyond traditional heuristic algorithms, researchers have increasingly turned to artificial intelligence techniques to address scheduling problems, such as reinforcement learning (RL). RL focuses on how an agent can maximize its rewards in complex and uncertain environments. By transforming cloud resources and job states into images that represent the system's state space, RL agents can maximize rewards according to their objectives and make scheduling decisions accordingly. The RL method has improved the generalization of scheduling decisions to some degree. Nevertheless, when confronted with diverse optimization goals or substantial variations in load conditions, retraining remains a necessity. Undoubtedly, the retraining process incurs additional training costs.

In summary, existing solutions for task scheduling still exhibit various limitations. Our exploration of LLM-based approaches may offer new directions and insights.

B. Large Language Models

1) *Fine-tuning and Prompting*: Currently, there are two main approaches for applying large-scale pre-trained foundation models to various downstream tasks: fine-tuning and prompting.

LLM fine-tuning is an approach that optimizes the parameters of a pre-trained model for a specific task. In the process, the existing parameters of the pre-trained model are used as the initial parameters, and then these parameters are updated through supervised training on a specific dataset. Although there are currently some parameter-efficient fine-tuning approaches that can reduce the number of parameters needing to be trained, the training process itself still requires considerable computational resources. For many institutions and individuals, obtaining sufficient computing power remains a challenge.

The emergence of the GPT-3 model has introduced a new way to apply LLMs to downstream tasks by suitable prompting. The core idea of prompting is to present task descriptions and examples in the form of natural language text, enabling LLMs to learn through analogy. Notably, prompting does not require any adjustment to the model parameters. Instead, it relies on carefully crafted prompting strategies to address the diverse challenges posed by downstream tasks.

Given the limited computational resources available to us, we have chosen the Prompting method for the experiments involving scheduling based on LLMs in this paper.

2) *Necessary tools*: In the realm of LLMs, Ollama and LangChain emerge as powerful tools for developers and researchers. Ollama provides a seamless way to run open-source LLMs locally, while LangChain offers a flexible framework for integrating these models into applications.

Ollama is a tool tailor-made for running and customizing large language models in local environments. It provides a straightforward and efficient interface for creating, running, and managing these models, accompanied by a rich library of pre-built models that can be seamlessly integrated into various applications. Ollama aims to simplify the deployment and interaction with large language models, catering to both developers and end-users alike. Additionally, Ollama supports a wide range of mainstream models available in the market, such as Llama, Qwen2, Mistral, Gemma, and more.

LangChain is a framework designed to facilitate the integration of LLMs into applications. It supports a wide range of chat models, including Ollama, and provides an expressive language for chaining operations. LLMChain is the most common type of chain, which comprises a PromptTemplate, a model, and an optional OutputParser. LLMChain accepts multiple input variables, formats them into prompts using the PromptTemplate, and then passes these prompts to the model. Finally, it utilizes the OutputParser to decode the output of the LLM into a final format.

3) *Well-known open-source LLMs*: A diverse range of open-source LLMs are now publicly accessible for research. For this study, we selected six prominent models from various organizations: Llama3 and Llama3.1 from Meta AI [15], Gemma2 from Google [16], the Qwen series from Alibaba [17], and Mistral [18]. To accommodate computational constraints and ensure reproducibility, we focused on smaller-scale variants of these models that can be executed on consumer-grade GPUs. Specifically, we experimented with Llama3-8B, Llama3.1-8B, Gemma2-9B, Qwen2-7B, Qwen2.5-7B, and Mistral-7B.

III. EXPERIMENT DESIGN

In this section, we introduce the design for online multi-resource cluster scheduling with LLMs. We formulate the problem (III-A) and describe how to infer it based on LLMs

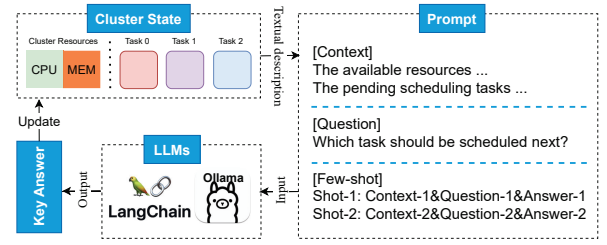


Fig. 1. Scheduling with LLMs.

(III-B). We then present the specific implementation of our experiments (III-C).

A. Model

Since we are exploring scheduling based on LLMs for the first time, we have designed our experimental scenarios to be both straightforward and effective. Specifically, in this paper we model the cluster environment with multiple types of resources, i.e. CPU and memory. Then we define an online scheduling process as selecting one or more tasks to execute from the online pending jobs at each timestep. We assume that the resource requirements and duration of each job are known upon arrival. Furthermore, we envision the cluster as a simple aggregation of resources, disregarding the complexities introduced by fragmentation. Besides, we assume that the scheduling is non-preemptive. These assumptions are deliberately chosen to simplify the scheduling model while preserving the fundamental aspects of multi-resource scheduling, thereby enabling us to explore the potential of LLM-based scheduling.

B. Scheduling with LLMs

The overall flow of LLM-based scheduling is shown in Fig. 1. First, the cluster state is dynamically extracted based on the environment configuration of the multi-resource online scheduling scenario. Specifically, the current available resources, job requirements, and job durations are gathered.

The numerical data are then transformed into natural language sentences to form the Context and few-shot examples for the Prompt. Here, two few-shot examples are set to strengthen the LLMs' understanding of the prompt. These Prompts are integrated into the PromptTemplate module of Langchain, which then invokes LLMs for inference through Ollama.

TABLE I
TEMPLATE FOR TRANSFORMING SCHEDULING TASKS TO NATURAL LANGUAGE DESCRIPTIONS.

		Template	Example
Input Prompt	Context	The available resources for the next 20 time steps are: $\{available_resources\}$ At this point, the pending scheduling tasks are: $\{pending_tasks\}$.	The available resources for the next 20 time steps are: [9. 1.] [9. 1.] [9. 1.] ... At this point, the pending scheduling tasks are: Task0: requiring resources of [10 2] and lasting for 13 time steps; Task1...
	Question	To minimize the $\{obj\}$, which task should be scheduled next?	To minimize the job slowdown, which task should be scheduled next?
Output Prompt	Answer	The next task to be scheduled is task $\{x\}$.	The next task to be scheduled is task 1.

Based on the LLM response, the key scheduling decision is extracted and the cluster state is updated accordingly, completing a single task scheduling cycle and constituting one iteration of the experiment.

The core task of LLM-based scheduling can be understood as inferring scheduling decisions through language-based outputs. To achieve the goal, the key step is to convert scheduling tasks into natural language descriptions. As shown in Table I, a template-based approach is used to transform data into text, aligning closely with the structure of the PromptTemplate module in Langchain.

The template consists primarily of two components: an input prompt and an output prompt. The input prompt is further divided into a context section and a question section. The context provides the environment state of the cluster, where $\{available_resources\}$ refers to the text derived from available resources, and $\{pending_tasks\}$ refers to the text representing the sequence of jobs to be scheduled, including their resource requirements and durations. The question section needs to specify the optimization objective, where $\{obj\}$ refers to either job slowdown or job completion time. The output prompt provides the basic format of the answer to the question. Here we also enhance the output format of LLMs with two few-shots, facilitating the subsequent extraction of key answers.

C. Implementation

1) *Workload*: The cluster is defined to include two types of resources, each with an available size of [10, 10]. Jobs arrive according to a Bernoulli process at a specified job arrival rate. Each job randomly selects one of the resources as its dominant resource, with a required size ranging from 5 to 10, while the requirement for the non-dominant resource ranges from 1 to 2. Jobs are classified as either long or short; long jobs have lengths between 10t and 15t, and short jobs have lengths between 1t and 3t. The length of the scheduling queue is set to 5, and the visibility time step for both resources is set to 20t. Each experiment lasts for 50t. The experiment ends when all jobs that arrived within the 50t period have been completed. Overall, it is to select a suitable task from a set of 5 pending tasks with known resource availability over a 20t time horizon to optimize the specified objective.

2) *Metrics*: We have chosen two metrics to assess the performance of different scheduling methods: the average job completion time and the average job slowdown.

- Job completion time: The time taken from the arrival of a job to its completion. Clearly, a shorter completion time is preferable.
- Job slowdown: Job slowdown measures job completion efficiency and is calculated as $S_j = \frac{C_j}{T_j}$, where C_j is the job completion time, and T_j denotes the ideal completion time (i.e., the job's length without waiting). Note that $S_j \geq 1$, with lower values indicating a better performance.

3) *Comparables*: The LLM-based scheduling strategy was compared with several classical scheduling methods:

- Random: The job is randomly selected from the pending job queue.
- SJF(Shortest Job First): Jobs are scheduled based on their length, with priority given to shorter jobs.
- Packer: Jobs are allocated based on the alignment between job requirements and resource availability [19].
- DeepRM: A DRL(Deep Reinforcement Learning)-based scheduler that requires pre-training for specific optimization objectives and workload [20]. In this study, the DeepRM model used for comparison was trained with the objective of minimizing the average job slowdown, and the proportion of short jobs in its training set was set at 80%.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

Based on the LLM-based scheduling approach outlined above, experiments were conducted with six popular open-source LLMs and we provide a comparative analysis of the experimental results.

A. Experimental setup

All experiments were conducted on a desktop computer with the following specifications: Ubuntu 20.04.6 LTS operating system, 13th Gen Intel® Core™ i9-13900F CPU, 64 GB RAM, NVIDIA GeForce RTX 4080 GPU with 16 GB of memory.

To manage workload (capped at 200%), the maximum job arrival rates were set as follows: 0.6 with a short job proportion of 80%, 0.5 with a short job proportion of 50%, and 0.4 with a short job proportion of 20%.

All metrics, including average job slowdown and average job completion time, represent the mean values from 100 experimental repetitions. The experimental dataset consisted of previously unseen tasks, not used during training of DeepRM.

B. Comparison of scheduling efficiency

Fig. 2 shows the average job slowdown for six open-source LLMs compared to other conventional scheduling methods at various job arrival rates. Here, the proportion of short jobs is set to 20%, matching the ratio used in the DeepRM training set, where the optimization objective is also the slowdown.

In general, the average slowdown increases with the job arrival rate, which aligns with expectations, as a higher arrival rate leads to a higher cluster load. In addition, the results indicate that the six open-source LLMs outperform the Packer and Random methods, but are weaker than SJF and DeepRM. Although LLMs do not achieve the best performance, they show promising potential for scheduling. It is worth noting that the LLMs relied solely on basic inference prompts. This finding gives us confidence that further prompt optimization or fine-tuning of the LLMs could yield more satisfactory results.

In addition, Fig. 2 indicates that Gemma2 performs best among the six LLMs, followed by the two models in the Llama series (Llama3 and Llama3.1), then the two models in the Qwen series (Qwen2 and Qwen2.5), with Mistral performing the worst. We observed that the ranking aligns with the

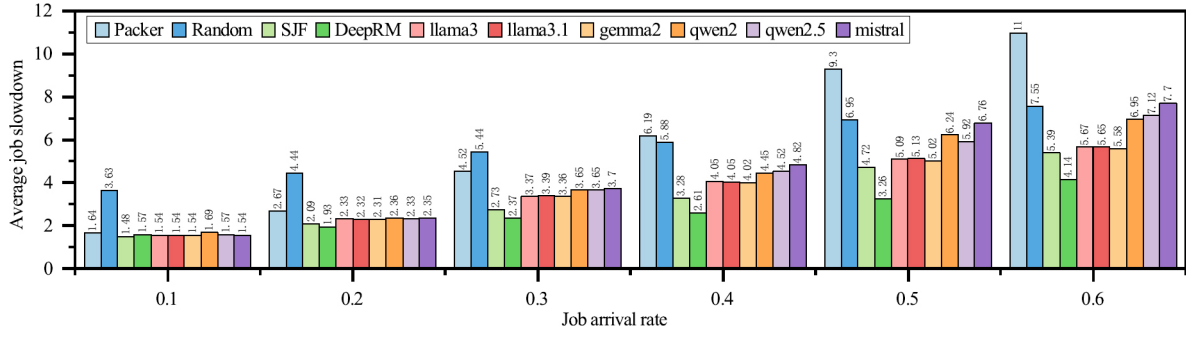


Fig. 2. Job slowdown at different job arrival rate (80% short job).

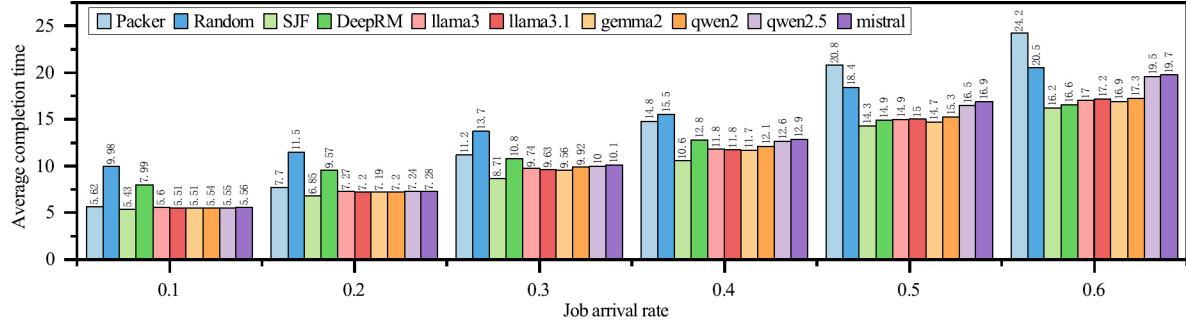


Fig. 3. Job completion time at different job arrival rate (80% short job).

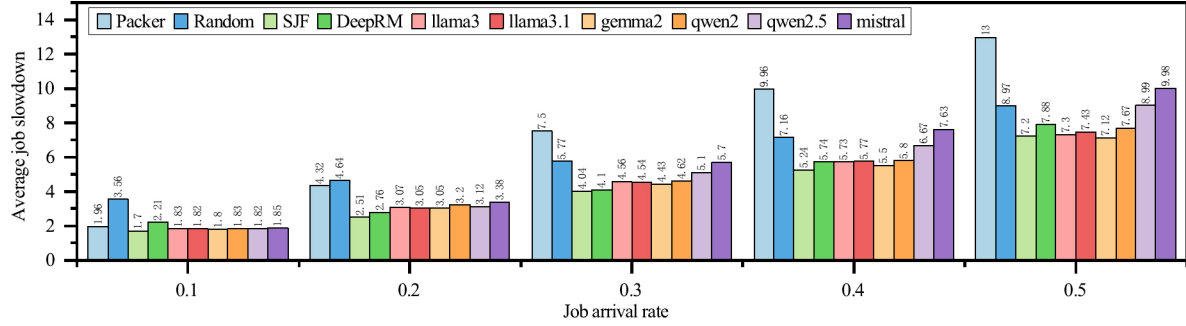


Fig. 4. Job slowdown at different job arrival rate (50% short job).

parameter scale of each model, which indirectly suggests a common trend in LLMs: As the parameter size increases, model performance tends to improve.

C. Comparison of generalization

The generalization of LLMs was investigated across different optimization objectives and short job ratios. Note that DeepRM here still refers to the RL model trained with job slowdown as the objective and a short-job ratio of 80%.

1) *Various objectives*: First, the optimization objective was set to job completion time, while keeping the short-job ratio constant. The experimental results are shown in Fig. 3. It is worth noting that for different optimization objectives, LLM-based scheduling requires only an adjustment to the

optimization objective ($\{obj\}$) within the prompt template, without the need for retraining as required by RL models.

As shown in Fig. 3, all LLMs outperform the DeepRM model when job arrival rates range from 0.1 to 0.4. Among them, Gemma2 yields the best results, with respective improvements over DeepRM of approximately 30%, 25%, 11%, and 9%. At higher job arrival rates, Gemma2 also maintains performance largely comparable to DeepRM. These results suggest that LLMs exhibit strong generalization across different optimization objectives.

2) *Various job distributions*: Next, we adjusted the proportion of short jobs to conduct comparative experiments, as shown in Fig. 4 and Fig. 5. Fig. 4 illustrates results for a 50% short job ratio, indicating that with the exception of

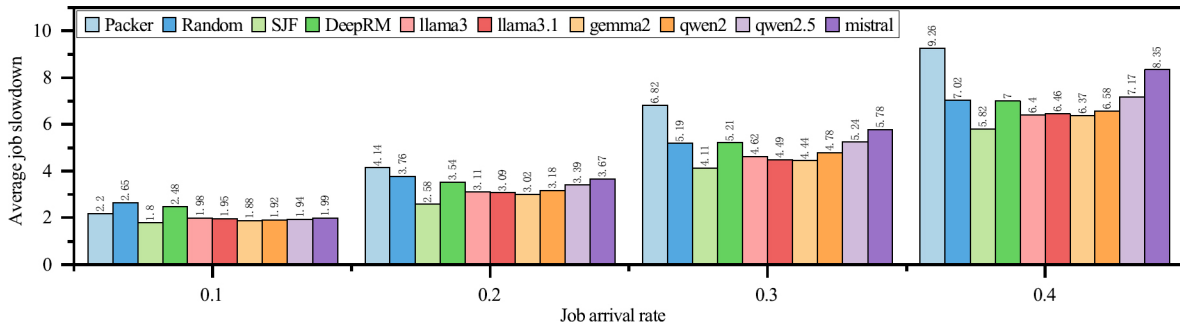


Fig. 5. Job slowdown at different job arrival rate (20% short job).

the Qwen2.5 and Mistral at the job arrival rate of 0.5, the performance of the LLMs is generally comparable to or even surpasses that of the DeepRM. In the case of a 20% short job (Fig. 5), nearly all LLMs significantly outperform DeepRM, suggesting that LLMs exhibit superior generalization across varying job distributions.

Furthermore, when considering the bar charts from Fig. 2 to Fig. 5, Gemma2 consistently demonstrates the best performance among the LLMs, while Mistral exhibits the poorest performance. Within the same model series, Llama3.1 generally outperforms Llama3, while Qwen2.5 underperforms relative to Qwen2. The discrepancy may stem from differences in the models' optimization focus.

V. CONCLUSION

We present an empirical study of task scheduling using LLMs. The experimental results indicate that LLMs hold substantial promise for a multi-resource cluster scheduling problem. In particular, LLMs demonstrate superior generalization compared to the RL model across various optimization objectives and workloads. Further optimization of prompts, model fine-tuning, or even pre-training may yield improved results in the future.

ACKNOWLEDGMENT

The work is supported by the National Natural Science Foundation of China under grant No. U22B2005 and No. 62372462, and is partially supported by the project of National Key Laboratory of Multi-domain Data Collaborative Processing and Control under grant No. MDPC20240202.

REFERENCES

- [1] Z. Han, C. Gao, J. Liu, S. Q. Zhang *et al.*, "Parameter-efficient fine-tuning for large models: A comprehensive survey," *arXiv preprint arXiv:2403.14608*, 2024.
- [2] T. B. Brown, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.
- [3] D. Dukić and J. Šnajder, "Looking right is sometimes right: Investigating the capabilities of decoder-only llms for sequence labeling," in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 14 168–14 181.
- [4] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang *et al.*, "chatie: Zero-shot information extraction via chatting with chatgpt," *arXiv preprint arXiv:2302.10205*, 2023.

- [5] L. Wang, C. Lyu, T. Ji, Z. Zhang, D. Yu, S. Shi, and Z. Tu, "Document-level machine translation with large language models," *arXiv preprint arXiv:2304.02210*, 2023.
- [6] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl *et al.*, "Large language models encode clinical knowledge," *Nature*, vol. 620, no. 7972, pp. 172–180, 2023.
- [7] H. Zhou, F. Liu, B. Gu, X. Zou, J. Huang, J. Wu, Y. Li, S. S. Chen, P. Zhou, J. Liu *et al.*, "A survey of large language models in medicine: Progress, application, and challenge," *arXiv preprint arXiv:2311.05112*, 2023.
- [8] F. Yu, L. Quaratey, and F. Schilder, "Legal prompting: Teaching a language model to think like a lawyer," *arXiv preprint arXiv:2212.01326*, 2022.
- [9] G. Son, H. Jung, M. Hahm, K. Na, and S. Jin, "Beyond classification: Financial reasoning in state-of-the-art language models," *arXiv preprint arXiv:2305.01505*, 2023.
- [10] A. Extance, "Chatgpt has entered the classroom: how llms could transform education," *Nature*, vol. 623, no. 7987, pp. 474–477, 2023.
- [11] C. Thapa, S. I. Jang, M. E. Ahmed, S. Camtepe, J. Pieprzyk, and S. Nepal, "Transformer-based language models for software vulnerability detection," in *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 481–496.
- [12] F. Li, H. Lang, J. Zhang, J. Shen, and X. Wang, "Preconfig: A pretrained model for automating network configuration," *arXiv preprint arXiv:2403.09369*, 2024.
- [13] H. Zhou, X. Ouyang, Z. Ren, J. Su, C. de Laat, and Z. Zhao, "A blockchain based witness model for trustworthy cloud service level agreement enforcement," in *IEEE INFOCOM 2019-IEEE conference on computer Communications*. IEEE, 2019, pp. 1567–1575.
- [14] M. Li, J. Su, H. Liu, Z. Zhao, X. Ouyang, and H. Zhou, "The extreme counts: modeling the performance uncertainty of cloud resources with extreme value theory," in *International Conference on Service-Oriented Computing*. Springer, 2022, pp. 498–512.
- [15] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [16] G. Team, M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé *et al.*, "Gemma 2: Improving open language models at a practical size," *arXiv preprint arXiv:2408.00118*, 2024.
- [17] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.
- [18] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. I. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.
- [19] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, "Multi-resource packing for cluster schedulers," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 455–466, 2014.
- [20] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proceedings of the 15th ACM workshop on hot topics in networks*, 2016, pp. 50–56.